



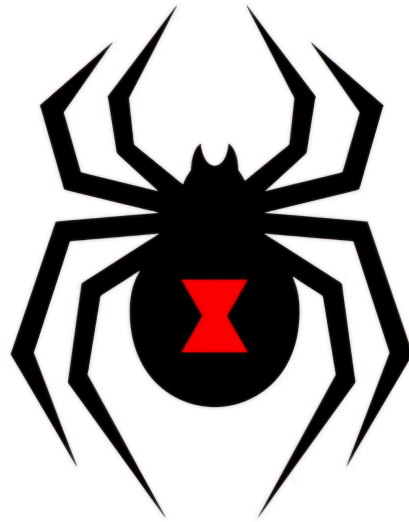
UNSA

Universidad Nacional de San Agustín

“AÑO DE LA ESPERANZA Y EL FORTALECIMIENTO DE LA DEMOCRACIA”

FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS

ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



VIUDA NEGRA

Plan de Pruebas del Sistema

ASIGNATURA:

Pruebas de Software

DOCENTE:

Ing. Robert Edison Arisaca Mamani

INTEGRANTES:

Tejada Lazo Jordy Rolando (Test Lead)

Hurtado Bejarano Michael Steve (Test Analyst)

Jaita Chura Jose Manuel (Test Design)

Yare Chulunquia Kevin Pedro (Test Analyst)

Del Castillo Montoya Christopher Brad (Architect)

Índice

Índice	2
1. Introducción	3
1.1. Alcance	3
1.2. Referencias	3
1.3. Glosario	4
2. Contexto de las Pruebas	5
2.1. Descripción del Proyecto InvenTree	5
2.2. Elementos de prueba (flujos de sistema)	6
2.3. Alcance de la prueba	7
2.4. Suposiciones y restricciones	8
2.5. Suposiciones y Restricciones	8
2.5.1. Suposiciones:	8
2.5.2. Restricciones:	8
2.6. Partes interesadas	9
3. Comunicación de las Pruebas	9
3.2. Protocolo de comunicación	10
4. Registro de Riesgos Técnicos	10
5. Estrategia de Prueba	12
5.1. Enfoque metodológico	12
5.2. Entorno y despliegue automatizado (CI/CD) para pruebas de sistema	12
5.3. Matriz de Casos de Prueba de Sistema (E2E)	13
5.4. Técnicas de diseño aplicadas	16
5.5. Criterios de aceptación del plan (nivel de proceso, no de negocio)	17
5.6. Métricas de prueba	17
5.7. Criterios de suspensión y reanudación	18
6. Actividades y Estimados de Prueba	18
7. Personal	20
7.1. Matriz RACI	20
7.2. Necesidades de contratación y capacitación	21

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 3
---	---------------------------------------

Versión	Autor(es)	Descripción	Fecha
1.0	Jaita Chura Jose Manuel	Versión inicial del documento	05/07/25

1. Introducción

1.1. Alcance

El presente documento constituye el Plan de Pruebas de Sistema del proyecto InvenTree. El plan se centra en dos atributos de calidad del sistema: Seguridad y Desempeño, seleccionados por ser los de mayor impacto sobre el sistema desplegado y los más verificables de forma automatizada dentro del tiempo disponible del curso.

A diferencia de las pruebas unitarias (caja blanca, ejecutadas por quien programa) y de las pruebas de integración (verifican la comunicación entre dos o más componentes internos ya construidos, por ejemplo API, ORM y base de datos), las pruebas de sistema de este plan evalúan preocupaciones transversales que atraviesan toda la superficie de la API del sistema desplegado, sin importar qué módulo de negocio esté detrás, desde una perspectiva de caja negra.

1.2. Referencias

- Repositorio de grupo:
https://github.com/jrolando19/VIUDA_NEGRA_PROJECT

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 4
---	---------------------------------------

- Repositorio de pruebas (fork de trabajo):
<https://github.com/jrolando19/InvenTree-VIUDANEGRA>
- Repositorio del proyecto original:
<https://github.com/inventree/InvenTree.git>
- GitHub Project (tablero Kanban/Scrum):
<https://github.com/users/jrolando19/projects/4>
- GitHub Pages del equipo:
https://jrolando19.github.io/VIUDA_NEGRA_PROJECT/
- GitHub Wiki del equipo:
https://github.com/jrolando19/VIUDA_NEGRA_PROJECT/wiki
- Documentación oficial InvenTree: <https://docs.inventree.org/>
- Documentación oficial — Desarrollo Frontend React / Testing E2E:
<https://docs.inventree.org/en/latest/develop/react-frontend/#testing>
- Workflow E2E : [.github/workflows/frontend.yaml](#)
- Workflow de construcción de imagen: [.github/workflows/docker.yaml](#)
- Workflow de importación/exportación de datos :
[.github/workflows/import_export.yaml](#)
- Workflow de chequeos de calidad : [.github/workflows/qc_checks.yaml](#)
- Actions del fork del equipo:
<https://github.com/jrolando19/InvenTree-VIUDANEGRA/actions>
- Carpeta de pruebas E2E del proyecto: `src/frontend/tests/` (archivos `pui_*.spec.ts`, patrón Page Object Model)
- Demo pública de referencia: <https://demo.inventree.org> (usuario admin / inventree)
- ISO/IEC/IEEE 29119 — Estándar internacional de pruebas de software
- Plantilla "Plan de Pruebas" del curso Pruebas de Software (UNSA-EPIS)

1.3. Glosario

- **PS (Prueba de Sistema):** Valida el sistema desplegado como un todo, desde la perspectiva de un usuario/cliente externo, sin acceder al código ni a la base de datos directamente.
- **Atributo de calidad (Quality Attribute):** Propiedad no funcional del sistema (seguridad, desempeño, concurrencia, disponibilidad, usabilidad)

que se evalúa de forma transversal, independiente de un flujo de negocio puntual.

- **Preocupación transversal (Cross-cutting concern):** Comportamiento que debe cumplirse en todos los módulos por igual (p. ej. "todo endpoint sin autenticación debe rechazar la petición"), y por eso se prueba replicando el mismo caso en múltiples módulos en vez de una sola vez.
- **Control de acceso:** Mecanismo que determina si una petición autenticada/no autenticada tiene permiso para ejecutar una acción sobre un recurso.
- **Control positivo:** Caso de prueba que verifica que la acción permitida sí funciona (p. ej. el admin sí puede leer), usado para descartar falsos positivos causados por un entorno caído.
- **Umbral de aceptación (Threshold):** Tiempo máximo, definido a priori, que un caso de desempeño tolera antes de considerarse una regresión.
- **Latencia p95:** Percentil 95 de una serie de mediciones de tiempo de respuesta; el 95% de las peticiones respondió en un tiempo igual o menor a ese valor.
- **Carga concurrente:** Conjunto de peticiones disparadas al mismo tiempo (en paralelo) contra el mismo endpoint, para detectar bloqueos o degradación bajo concurrencia.
- **Stock Item:** Instancia física/lote de una parte, con cantidad y ubicación.
- **Datos semilla (seed data):** Conjunto de datos de demostración cargado antes de las pruebas para tener un estado inicial conocido y reproducible.

2. Contexto de las Pruebas

2.1. Descripción del Proyecto InvenTree

InvenTree es un sistema open-source maduro de gestión de inventarios enfocado en el control de stock de bajo nivel y la trazabilidad detallada de partes y componentes.

Atributo	Detalle técnico Basal
Versión bajo Análisis	v1.3.0 (Release: Abril 2026)
Licencia	MIT License
Arquitectura Backend	Python 3.11 / 3.14 + Django 5.x + Django REST Framework (DRF)
Arquitectura Frontend	React + TypeScript + Mantine UI + TanStack Query

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 6
---	---------------------------------------

Motores de Persistencia	SQLite (entorno local/test), PostgreSQL 17 / MySQL 9 (entornos productivos)
Mapeo de Tareas Async	Django Q2 (Manejo en segundo plano de reportes y notificaciones automáticas)
Orquestador de CI/CD	GitHub Actions mediante el script unificado qc_checks.yaml (681 líneas)
Métricas de Cobertura	~87% combinado reportado oficialmente en el badge de Codecov
Métricas del Repositorio	+7.1k estrellas, 1.4k forks y 17,657 commits históricos

2.2. Atributos de Calidad Evaluados

El elemento de prueba de este plan es el atributo de calidad: cada caso verifica una propiedad no funcional a través de toda la API, no un recorrido de negocio puntual.

Atributo	Qué se verifica en este plan	Alcance transversal (módulos/endpoints)	Por qué se eligió
Seguridad	Que el control de acceso (autenticación y autorización) se cumpla de forma consistente en toda la superficie de la API, sin importar el módulo de negocio detrás.	part, stock, order/po, order/so, build, company, user, plugins	Es el atributo de mayor impacto si falla: un solo endpoint desprotegido compromete todo el sistema, no solo un flujo.
Desempeño	Tiempos de respuesta de endpoints críticos contra umbrales de aceptación, y comportamiento bajo carga concurrente ligera	part (lectura), stock (escritura individual y en lote)	Detecta regresiones reales (endpoint colgado, N+1 queries) sin requerir infraestructura de pruebas de carga de producción,

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 7
---	---------------------------------------

	y operaciones en lote.		fuera del alcance del curso.
--	------------------------	--	------------------------------

2.3. Alcance de la prueba

2.3.1. Incluido en el alcance :

- Diseño y ejecución de 2 suites de pruebas de sistema automatizadas, basadas en atributos de calidad transversales: Seguridad (SEC-01 a SEC-08) y Desempeño (PERF-01 a PERF-05).
- Ejecución de ambas suites contra la API REST del sistema desplegado (no contra el código fuente ni la base de datos directamente): sigue siendo caja negra, pero el punto de observación es el límite de la API en vez de la interfaz gráfica.
- Verificación de control de acceso (autenticación, autorización, prevención de escalación de privilegios) en los 7 módulos principales de la API: part, stock, order/po, order/so, build, company, user, y en el endpoint de plugins.
- Medición de tiempos de respuesta, latencia p95 y comportamiento bajo carga concurrente ligera (10 requests simultáneas) y en lote (20 StockItems) sobre endpoints críticos.
- Despliegue local del entorno (Docker/WSL2) como base estable y reproducible para ejecutar ambas suites.
- Documentación técnica de la estrategia y matrices de casos, suficiente para que el plan pueda ser re-ejecutado y reportado por otro integrante del equipo en el Informe de Pruebas de Sistema.

2.3.2. Excluido del alcance:

- Pruebas de concurrencia a gran escala, disponibilidad y usabilidad: no forman parte de este plan, que se centra en Seguridad y Desempeño.
- Pruebas de carga y estrés de producción (miles de usuarios concurrentes, infraestructura dedicada): la carga concurrente evaluada en PERF-03 (10 requests simultáneas) es una validación ligera de comportamiento bajo concurrencia, no un test de estrés real.
- Pruebas de penetración / auditoría de seguridad avanzada (OWASP Top 10 completo, fuzzing, inyección de código): SEC-01 a SEC-08 cubren control de acceso (autenticación/autorización), no vulnerabilidades a nivel de código.
- Pruebas funcionales de flujos de negocio de extremo a extremo vía UI ("Golden Path": compra → fabricación → venta) — se cubrieron en un entregable de pruebas de integración aparte (Hito 3), no forman parte de este plan de atributos de calidad.

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 8
---	---------------------------------------

- Integración con servicios reales de producción (SMTP real, pasarelas de pago, proveedores OAuth reales).

2.4. Suposiciones y restricciones

2.4.1. Suposiciones:

El fork InvenTree-VIUDANEGRA puede desplegarse de forma estable en un entorno local mediante Docker/WSL2, accesible en <http://localhost:8000/api>.

- Existe un usuario admin (admin / inventree) con permisos completos, usado como control positivo y como cuenta administradora en las suites.
- Existe un usuario testviewer (viewer123) sin permisos de superusuario (is_superuser=False), creado en el dataset y usado como control negativo en todos los casos de seguridad.
- La parte con pk=1 ("Resistencia 10k") existe en el dataset semilla y se usa como parte de referencia en ambas suites.
- El dataset de demostración (invoke dev.setup-test -iv) es representativo y suficiente para ejercitar los endpoints evaluados.

2.4.2. Restricciones:

- El equipo no tiene acceso a un entorno de producción real; el entorno de sistema se simula con Docker en un entorno controlado (local, vía WSL2).
- Los umbrales de desempeño (PERF-01, PERF-02, PERF-05: 2.0s / 1.0s por ítem) son deliberadamente generosos: el objetivo es detectar una regresión real sobre un servidor de desarrollo sin optimizar, no medir performance de producción.
- La carga concurrente evaluada (10 requests) está limitada por la capacidad del entorno local de desarrollo, no representa el volumen de un entorno productivo.

2.5. Partes interesadas

Rol	Integrante	Responsabilidades
Docente	Ing. Robert Edison Arisaca Mamani	Valida el plan de pruebas de sistema y sus entregables.
Test Lead	Ejada Lazo, Jordy Rolando	Coordina el equipo, gestiona riesgos,

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 9
---	---------------------------------------

		aprueba entregables de prueba.
Test Analyst	Hurtado Bejarano, Michael Steve	Diseña y ejecuta casos de prueba de sistema, reporta defectos.
Test Analyst	Yare Chulunquia, Kevin Pedro	Diseña y ejecuta casos de prueba de sistema, reporta defectos.
Test Architect	Del Castillo Montoya, Christopher Brad	Define la arquitectura de pruebas E2E, configura el despliegue CI/CD, selecciona herramientas.
Test Design	Jaita Chura, Jose Manuel	Diseña la matriz de casos de sistema, redacta y publica el plan en el Wiki.

3. Comunicación de las Pruebas

3.1. Objetivo

Asegurar que el equipo y el docente conozcan en todo momento el estado del entorno de pruebas de sistema, el avance de diseño/ejecución de casos E2E, los defectos encontrados y el estado del despliegue CI/CD.

3.2. Protocolo de comunicación

Punto de comunicación	Propósito	Frecuencia	Canal	Responsable
Kick-off Hito 3	Presentar el alcance del plan de sistema y asignar responsables por flujo (F1-F10).	Una vez	Google Meet	Test Lead

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 10
---	--

Stand-up	Reporte de avance por flujo/caso de prueba.	2-3 veces/semana	WhatsApp	Test Lead
Notificación de GitHub Actions	Alerta de fallo en el job de despliegue o en Playwright.	Cada push/PR	GitHub Actions	Automático
Registro de defecto	Documentar fallo funcional encontrado en un flujo de sistema.	Al encontrar un problema	GitHub Issues (label: system-test)	Test Analyst
Actualización del Wiki	Publicar este plan y, luego, el informe de resultados.	Una vez por documento	GitHub Wiki	Test Design
Sustentación Hito 3	Entrega final: demo del despliegue en vivo + resultados de sistema.	08-Jul-2026	Presencial/Meet	Todo el equipo

4. Registro de Riesgos Técnicos

Nº	Riesgo	Evidencia	Prob	Imp	Sev	Mitigación / Contingencia
1	El entorno local (Docker/WSL2) no levanta de forma estable, bloqueando la ejecución de ambas suites.	Actions del fork: Docker #1 (f10ccf8), #2 (b00d846), #3 (3e3ac77) — las 3 en rojo	3	4	12	Documentar el procedimiento de levantamiento paso a paso; usar docker-compose local como entorno de respaldo si el pipeline de Actions falla.
2	El usuario testviewer usado como control negativo termina con	Actions del fork: Frontend #1 (23-Jun,	2	4	8	Verificar explícitamente is_superuser=False antes de cada corrida; SEC-03

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 11
---	--

	permisos elevados por error de configuración del dataset, invalidando los casos SEC-01, SEC-02, SEC-06 y SEC-07.	b00d846), Frontend #2 x (hoy, 3e3ac77)				actúa como control positivo cruzado para detectar un entorno mal configurado.
3	Los umbrales de desempeño (2.0s / 1.0s por ítem) resultan demasiado ajustados para el hardware local del equipo, generando falsos fallos en PERF-01, PERF-02 o PERF-05.	—	3	2	6	Umbrales definidos deliberadamente generosos; si se repiten fallos, documentar el hardware usado y ajustar el umbral con justificación en el informe.
4	Tiempo insuficiente antes del cierre del hito para cubrir los 13 casos (SEC-01..08, PERF-01..05).	Tiempos reales observados en Actions: Frontend #1 = 20m 9s, Frontend #2 = 23m 36s, Docker = 39-40 min por corrida	3	4	12	Priorizar SEC-04 (transversal, 7 módulos) y PERF-01/PERF-04 (los más representativos) si el tiempo se reduce; el resto se documenta como pendiente/parcial.
5	Defectos reales de InvenTree (proyecto de terceros) bloquean un caso y no pueden corregirse.	—	2	3	6	Documentar el defecto como hallazgo (GitHub Issue), continuar con datos/rutas alternativas, no modificar el código fuente.
6	Falta de disponibilidad de algún integrante en la semana de cierre.	—	3	3	9	Distribución clara de casos (SEC vs. PERF) entre los integrantes desde el inicio del sprint; RACI claro (sección 7).

5. Estrategia de Prueba

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 12
---	--

5.1. Enfoque metodológico

- Caja negra, dirigido por atributos de calidad: el plan se centra en Seguridad y Desempeño, por ser los atributos de mayor impacto sobre el sistema desplegado y los más verificables de forma automatizada dentro del tiempo disponible.
- Transversal, no por módulo ni por flujo de negocio: cada caso evalúa una preocupación que atraviesa toda la superficie de la API (por ejemplo, si todo endpoint sin autenticación es rechazado), sin importar qué módulo de negocio esté detrás.
- Ejecutado vía API REST del sistema desplegado: las suites usan la librería requests de Python contra `http://localhost:8000/api`, sobre una instancia InvenTree desplegada en Docker/WSL2. Sigue siendo caja negra, ya que no se invoca código interno ni se lee la base de datos directamente; el punto de observación es el límite de la API y no la interfaz gráfica, pues el control de acceso y los tiempos de respuesta se miden de forma más precisa y repetible ahí.
- Automatizado de punta a punta: ambas suites (`test_sistema_seguridad_inventree_suite.py`, `test_sistema_desempeno_inventree_suite.py`) corren sin intervención manual, registran resultados estructurados en JSON (`test_output/results/`) y retornan código de salida distinto de cero si hay algún fallo, listas para integrarse a CI/CD.
- Control positivo incluido: cada suite incorpora al menos un caso de control (p. ej. SEC-03: el admin sí puede leer con normalidad) para descartar falsos positivos causados por un entorno caído en vez de por una regla de seguridad real.

5.2. Entorno y despliegue automatizado (CI/CD) para pruebas de sistema

Etapas	Herramienta (ya presente en el fork)	Descripción	Estado actual
Provisioning de datos	<code>invoke dev.setup-test -iv</code>	Migra la BD y carga el dataset de demostración, incluyendo el usuario <code>testviewer</code> sin permisos.	Funcional.
Despliegue del entorno	Docker / WSL2 (backend + frontend)	Levanta el stack InvenTree completo y aislado, accesible en <code>http://localhost:8000/api</code> .	Funcional en entorno local.
Ejecución — Seguridad	<code>test_sistema_seguridad_inventree_suite.py</code> (requests)	Corre SEC-01 a SEC-08 contra el stack desplegado; usa	Implementada y ejecutable.

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 13
---	--

		AUTH_ADMIN y AUTH_USER (testviewer).	
Ejecución — Desempeño	test_sistema_desempeno_inventree_suite.py (requests, ThreadPoolExecutor, statistics)	Corre PERF-01 a PERF-05: tiempos individuales, carga concurrente (10 hilos), p95 y lote (20 ítems).	Implementada y ejecutable.
Reporte	JSON estructurado (seguridad_results.json, desempeno_results.json)	Resultado por caso (PASS/FAIL, detalle) y resumen de conteo; insumo directo para el Informe de Pruebas de Sistema.	Disponible tras cada corrida.

5.3. Matriz de Casos de Prueba de Sistema - Seguridad

ID	Flujo	Escenario	Precondición	Pasos clave	Resultado esperado	Automatización
SEC-01	Permisos — Partes	Usuario testviewer autenticado (sin permisos)	DELETE /api/part/1/ con credenciales de testviewer	HTTP 401/403 — eliminación rechazada	SEC-01	Permisos — Partes
SEC-02	Permisos — Partes	Usuario testviewer autenticado	POST /api/part/ creando una parte nueva con testviewer	HTTP 401/403 — creación rechazada	SEC-02	Permisos — Partes
SEC-03	Control positivo	Usuario admin autenticado	GET /api/part/1/ con admin	HTTP 200 y pk correcto — el admin sí puede leer con normalidad	SEC-03	Control positivo
SEC-04	Autenticación transversal	Ninguna (sin token)	GET sin autenticación a 7	HTTP 401/403 en los 7	SEC-04	Autenticación transversal

			módulos: part, stock, order/po, order/so, build, company, user	módulos evaluados		
SEC-05	Validación de token	Ninguna	GET /api/user/m e/ con: (a) token inexistente, (b) token vacío, (c) esquema Bearer no soportado	HTTP 401 en los 3 casos	SEC-05	Validación de token
SEC-06	Permisos — Stock	Usuario testviewer / StockItem creado previament e por admin	(a) POST /api/stock/ con testviewer (b) DELETE de ese StockItem con testviewer	HTTP 401/403 en ambos casos	SEC-06	Permisos — Stock
SEC-07	Escalación de privilegios	Usuario testviewer autenticado	(a) POST /api/user/ creando otro usuario (b) PATCH /api/user/1/ con is_superuser=True	HTTP 401/403/40 4 — no puede crear usuarios ni auto-otorga rse superuser	SEC-07	Escalación de privilegios
SEC-08	Autenticaci ón — Plugins	Ninguna	GET /api/plugin s/ sin autenticaci ón	HTTP 401/403	SEC-08	Autenticaci ón — Plugins

5.4. Matriz de Casos de Prueba de Sistema - Desempeño

Mide tiempo de respuesta de endpoints críticos contra umbrales de aceptación y comportamiento bajo carga; no valida reglas de negocio, solo tiempos.

ID	Flujo	Escenario	Precondición	Pasos clave	Resultado esperado	Automatización
PERF-01	Tiempo de respuesta — lectura	GET /api/part/?limit=100	HTTP 200 dentro del umbral	< 2.0 s	PERF-01	Tiempo de respuesta — lectura
PERF-02	Tiempo de respuesta — escritura	POST /api/stock/ (crear 1 StockItem)	HTTP 201 dentro del umbral	< 2.0 s	PERF-02	Tiempo de respuesta — escritura
PERF-03	Carga concurrente	10 GET /api/part/?limit=20 simultáneos (ThreadPoolExecutor, 10 hilos)	Las 10 peticiones exitosas (200) y tiempo total dentro del umbral	< 10.0 s total	PERF-03	Carga concurrente
PERF-04	Latencia p95	20 GET /api/part/?limit=20 secuenciales; se calcula el percentil 95	0 fallos y p95 dentro del umbral	p95 < 1.5 s	PERF-04	Latencia p95
PERF-05	Rendimiento en lote	Crear 20 StockItems de forma secuencial vía POST /api/stock/	0 fallos y tiempo promedio por ítem dentro del umbral	< 1.0 s/ítem	PERF-05	Rendimiento en lote

5.4.1. Técnicas de diseño aplicadas

- Pruebas de control de acceso negativo y positivo: por cada regla de permisos se valida tanto que un usuario sin privilegios sea rechazado (negativo, p. ej. SEC-01/02/06/07) como que un usuario autorizado sí pueda operar (positivo, SEC-03), evitando falsos positivos.

- Pruebas transversales por preocupación (cross-cutting concern testing): SEC-04 replica la misma verificación (rechazo sin autenticación) en 7 módulos distintos, para detectar módulos que hayan quedado desprotegidos por configuración.
- Pruebas basadas en umbral (threshold-based testing): PERF-01, PERF-02 y PERF-05 comparan el tiempo medido contra un umbral fijo definido a priori.
- Pruebas de carga concurrente ligera: PERF-03 dispara 10 peticiones simultáneas con ThreadPoolExecutor para detectar bloqueos o degradación básica bajo concurrencia, sin construir infraestructura de pruebas de carga de producción.
- Análisis de percentiles: PERF-04 calcula el p95 sobre 20 muestras, una métrica más robusta que el promedio frente a valores atípicos (outliers).

5.5. Criterios de aceptación del plan (nivel de proceso, no de negocio)

Estos criterios evalúan si este plan de pruebas de sistema está completo y es ejecutable; no deben confundirse con los criterios de negocio del Plan de Pruebas de Aceptación (documento aparte).

Criterio	Condición de éxito	Prioridad
Todos los casos SEC-01 a SEC-08 y PERF-01 a PERF-05 diseñados y documentados	13/13 casos con precondition, método y resultado esperado definidos	Alta
Ambas suites ejecutables de forma automatizada y repetible	Ambos scripts corren sin intervención manual y generan JSON con PASS/FAIL por caso	Crítica
Entorno de sistema desplegado localmente (Docker/WSL2), reproducible	Stack completo disponible en localhost:8000 antes de cada corrida	Alta
Trazabilidad de defectos de sistema	Todo fallo encontrado durante el diseño/piloto documentado en GitHub Issues (label: system-test)	Media

5.6. Métricas de prueba

Métrica	Cómo medirla	Valor objetivo
Casos de sistema ejecutados / diseñados	$(\text{SEC} + \text{PERF ejecutados} / 13) \times 100\%$	100% en el informe final
Cobertura de control de acceso	Módulos verificados sin autenticación (SEC-04) / módulos totales de la API	7/7 módulos (100%)
Cumplimiento de umbrales de desempeño	Casos PERF que cumplen su umbral / 5 totales	100% (o justificar cada excepción)
Defectos de sistema encontrados y documentados	Conteo de GitHub Issues con label system-test	Registrar el 100% de los encontrados

5.7. Criterios de suspensión y reanudación

- Suspensión: las pruebas se detienen si el entorno desplegado (Docker/WSL2) no responde en /api/ por más de dos intentos consecutivos, o si el caso de control positivo SEC-03 falla, indicaría un entorno mal configurado y no una falla de seguridad real.
- Reanudación: se reanuda cuando el Test Architect confirma que el entorno responde correctamente (SEC-03 pasa) y el Test Lead aprueba continuar.

6. Actividades y Estimados de Prueba

A continuación, se describen las actividades principales planificadas para el desarrollo del plan de pruebas unitarias:

ID	Actividad	Responsable	Hrs. Est.	Descripción
A-01	Definir estrategia y alcance del plan de pruebas de sistema, centrado en Seguridad y Desempeño	Jaita Chura, Jose Manuel	2 hrs	Redactar las secciones 1.1, 2.2 y 2.3 de este documento.

INVENTREE Plan de Pruebas de Sistema	UNSA - EPIS Página 18
---	--

A-02	Diseñar matriz de casos de Seguridad (SEC-01 a SEC-08)	Jaita Chura, Jose Manuel + Hurtado Bejarano, Michael Steve	4 hrs	Definir precondition, método y resultado esperado de cada caso de control de acceso.
A-03	Diseñar matriz de casos de Desempeño (PERF-01 a PERF-05)	Jaita Chura, Jose Manuel + Yare Chullunquia, Kevin Pedro	3 hrs	Definir método, umbral y resultado esperado de cada caso de tiempos/carga.
A-04	Configurar usuario testviewer (sin permisos) y entorno local Docker/WSL2	Del Castillo Montoya, Christopher Brad	3 hrs	Preparar el entorno estable para que ambas suites puedan ejecutarse de forma repetible.
A-05	Implementar test_sistema_seguridad_inventree_suite.py	Hurtado Bejarano, Michael Steve + Jaita Chura, Jose Manuel	5 hrs	Automatizar SEC-01 a SEC-08 contra el entorno desplegado.
A-06	Implementar test_sistema_desempeno_inventree_suite.py	Yare Chullunquia, Kevin Pedro + Del Castillo Montoya, Christopher Brad	5 hrs	Automatizar PERF-01 a PERF-05, incluyendo carga concurrente y cálculo de p95.
A-07	Ejecutar ambas suites y registrar resultados	Todo el equipo	2 hrs	Corrida de verificación end-to-end; revisar los JSON generados antes de la entrega.
A-08	Consolidar y publicar el Plan	Jaita Chura, Jose Manuel	2 hrs	Formatear y subir este

	de Pruebas de Sistema			documento al Wiki del equipo.
--	-----------------------	--	--	-------------------------------

7. Personal

7.1. Matriz RACI

R: Responsable | A: Aprobador | C: Consultado | I: Informado

Actividad	Docente	Ejada Lazo, Jordy (Test Lead)	Hurtado Bejarano, Michael (Test Analyst)	Yare Chulunqui, Kevin (Test Analyst)	Del Castillo Montoya, Christopher (Architect)	Jaita Chura, Jose (Design)
A-01 Estrategia y alcance	I	A	C	C	C	R
A-02 Matriz Seguridad	I	A	R	C	I	R
A-03 Matriz Desempeño	I	A	C	R	I	R
A-04 Entorno y usuario de prueba	I	A	I	I	R	C
A-05 Suite Seguridad	I	A	R	I	C	R
A-06 Suite Desempeño	I	A	I	R	R	C

A-07 Ejecución y resultados	I	R	R	R	R	R
A-08 Publicación del plan	I	A	I	I	I	R

7.2. Necesidades de contratación y capacitación

No se requiere personal adicional. Se recomienda una breve capacitación interna (0.5-1 hora) sobre: (a) cómo levantar el entorno local vía Docker/WSL2 y crear el usuario testviewer sin permisos, y (b) cómo ejecutar ambas suites (python `test_sistema_seguridad_inventree_suite.py` / `test_sistema_desempeno_inventree_suite.py`) y leer los resultados JSON, a cargo de Del Castillo Montoya, Christopher Brad (Test Architect), dirigida a quienes ejecuten las suites en el siguiente entregable.